

# AG35-CET&AG35-EUT

## QuecOpen(SDK) SPI 开发指导

LTE Standard 模块系列

版本: 1.0

日期: 2024-11-29

状态: 受控文件



上海移远通信技术股份有限公司（以下简称“移远通信”）始终以为客户提供最及时、最全面的服务为宗旨。如需任何帮助，请随时联系我司上海总部，联系方式如下：

上海移远通信技术股份有限公司

上海市闵行区田林路 1016 号科技绿洲 3 期（B 区）5 号楼 邮编：200233

电话：+86 21 5108 6236 邮箱：[info@quectel.com](mailto:info@quectel.com)

或联系我司当地办事处，详情请登录：<http://www.quectel.com/cn/support/sales.htm>。

如需技术支持或反馈我司技术文档中的问题，请随时登录网址：

<http://www.quectel.com/cn/support/technical.htm> 或发送邮件至：[support@quectel.com](mailto:support@quectel.com)。

## 前言

移远通信提供该文档内容以支持客户的产品设计。客户须按照文档中提供的规范、参数来设计产品。同时，您理解并同意，移远通信提供的参考设计仅作为示例。您同意在设计您目标产品时使用您独立的分析、评估和判断。在使用本文档所指导的任何硬软件或服务之前，请仔细阅读本声明。您在此承认并同意，尽管移远通信采取了商业范围内的合理努力来提供尽可能好的体验，但本文档和其所涉及服务是在“可用”基础上提供给您的。移远通信可在未事先通知的情况下，自行决定随时增加、修改或重述本文档。

## 使用和披露限制

### 许可协议

除非移远通信特别授权，否则我司所提供硬软件、材料和文档的接收方须对接收的内容保密，不得将其用于除本项目的实施与开展以外的任何其他目的。

### 版权声明

移远通信产品和本协议项下的第三方产品可能包含受移远通信或第三方材料、硬软件和文档版权保护的相关资料。除非事先得到书面同意，否则您不得获取、使用、向第三方披露我司所提供的文档和信息，或对此类受版权保护的资料进行复制、转载、抄袭、出版、展示、翻译、分发、合并、修改，或创造其衍生作品。移远通信或第三方对受版权保护的资料拥有专有权，不授予或转让任何专利、版权、商标或服务商标权的许可。为避免歧义，除了正常的非独家、免版税的产品使用许可，任何形式的购买都不可被视为授予许可。对于任何违反保密义务、未经授权使用或以其他非法形式恶意使用所述文档和信息的违法侵权行为，移远通信有权追究法律责任。

### 商标

除另行规定，本文档中的任何内容均不授予在广告、宣传或其他方面使用移远通信或第三方的任何商标、商号及名称，或其缩略语，或其仿冒品的权利。

### 第三方权利

您理解本文档可能涉及一个或多个属于第三方的硬软件和文档（“第三方材料”）。您对此类第三方材料的使用应受本文档的所有限制和义务约束。

移远通信针对第三方材料不做任何明示或暗示的保证或陈述，包括但不限于任何暗示或法定的适销性或特定用途的适用性、平静受益权、系统集成、信息准确性以及与许可技术或被许可人使用许可技术相关的不侵犯任何第三方知识产权的保证。本协议中的任何内容都不构成移远通信对任何移远通信产品或任何其他硬件、设备、工具、信息或产品的开发、增强、修改、分销、营销、销售、提供销售或以其他方式维持生产的陈述或保证。此外，移远通信免除因交易过程、使用或贸易而产生的任何和所有保证。

## 隐私声明

为实现移远通信产品功能，特定设备数据将会上传至移远通信或第三方服务器（包括运营商、芯片供应商或您指定的服务器）。移远通信严格遵守相关法律法规，仅为实现产品功能之目的或在适用法律允许的情况下保留、使用、披露或以其他方式处理相关数据。当您与第三方进行数据交互前，请自行了解其隐私保护和数据安全政策。

## 免责声明

- 1) 移远通信不承担任何因未能遵守有关操作或设计规范而造成损害的责任。
- 2) 移远通信不承担因本文档中的任何因不准确、遗漏、或使用本文档中的信息而产生的任何责任。
- 3) 移远通信尽力确保开发中功能的完整性、准确性、及时性，但不排除上述功能错误或遗漏的可能。除非另有协议规定，否则移远通信对开发中功能的使用不做任何暗示或法定的保证。在适用法律允许的最大范围内，移远通信不对任何因使用开发中功能而遭受的损害承担责任，无论此类损害是否可以预见。
- 4) 移远通信对第三方网站及第三方资源的信息、内容、广告、商业报价、产品、服务和材料的可访问性、安全性、准确性、可用性、合法性和完整性不承担任何法律责任。

版权所有 ©上海移远通信技术股份有限公司 2024，保留一切权利。

**Copyright © Quectel Wireless Solutions Co., Ltd. 2024.**

# 文档历史

## 修订记录

| 版本  | 日期         | 作者           | 变更表述                                           |
|-----|------------|--------------|------------------------------------------------|
| -   | 2024-03-08 | Aurora JIANG | 文档创建                                           |
| 1.0 | 2024-11-29 | Gabriel LI   | 受控版本<br>更新加载 SPI 设备驱动的部分参数及默认值（第 5 章、第 6.2 章）。 |

## 目录

|                              |           |
|------------------------------|-----------|
| 文档历史 .....                   | 3         |
| 目录 .....                     | 4         |
| 表格索引 .....                   | 5         |
| 图片索引 .....                   | 6         |
| <b>1 引言 .....</b>            | <b>7</b>  |
| <b>2 SPI 说明 .....</b>        | <b>8</b>  |
| <b>3 引脚定义及使用 .....</b>       | <b>10</b> |
| 3.1. 引脚定义 .....              | 10        |
| 3.2. 引脚使用 .....              | 11        |
| 3.2.1. 标准 4 线 SPI 定义 .....   | 11        |
| 3.2.2. 扩展 6 线 SPI 使用 .....   | 12        |
| 3.2.2.1. 引脚使用 .....          | 12        |
| 3.2.2.2. 请求发起流程 .....        | 12        |
| <b>4 设备树配置 .....</b>         | <b>14</b> |
| 4.1. SPI1 设备树配置 .....        | 14        |
| 4.1.1. SPI1 总线设备树配置 .....    | 14        |
| 4.1.2. SPI1 引脚设备树配置 .....    | 15        |
| 4.2. SPI2 设备树配置 .....        | 16        |
| 4.2.1. SPI2 总线设备树配置 .....    | 16        |
| 4.2.2. SPI2 引脚设备树配置 .....    | 17        |
| 4.3. 复用 SPI 设备树配置 .....      | 17        |
| <b>5 SPI 设备驱动 .....</b>      | <b>19</b> |
| 5.1. 标准 4 线 SPI 设备驱动 .....   | 19        |
| 5.2. 扩展 6 线 SPI 设备驱动 .....   | 20        |
| <b>6 编译示例及功能验证 .....</b>     | <b>23</b> |
| 6.1. 编译步骤 .....              | 23        |
| 6.1.1. 标准 4 线 SPI 编译步骤 ..... | 23        |
| 6.1.2. 扩展 6 线 SPI 编译步骤 ..... | 24        |
| 6.2. 功能验证 .....              | 24        |
| 6.2.1. SPI1 主模式功能验证 .....    | 24        |
| 6.2.2. SPI2 主模式功能验证 .....    | 25        |
| 6.2.3. SPI 从模式功能验证 .....     | 27        |
| 6.2.4. 6 线 SPI 功能验证 .....    | 30        |
| <b>7 驱动调试 .....</b>          | <b>32</b> |
| <b>8 附录 参考文档及术语缩写 .....</b>  | <b>33</b> |

## 表格索引

|                                          |    |
|------------------------------------------|----|
| 表 1: SPI1（软件设备树节点：ssp1）引脚定义.....         | 10 |
| 表 2: SPI2（软件设备树节点：ssp0）引脚定义.....         | 10 |
| 表 3: 复用 SPI（软件设备树节点：ssp2）引脚定义.....       | 11 |
| 表 4: SPI1（软件设备树节点：ssp1）引脚定义.....         | 11 |
| 表 5: 扩展 6 线 SPI 引脚功能描述 .....             | 12 |
| 表 6: 内核模块加载时支持的参数（标准 4 线 SPI 设备驱动） ..... | 19 |
| 表 7: 内核模块加载时支持的参数（扩展 6 线 SPI 设备驱动） ..... | 21 |
| 表 8: 参考文档 .....                          | 33 |
| 表 9: 术语缩写 .....                          | 33 |

Quectel  
Confidential

## 图片索引

|                      |    |
|----------------------|----|
| 图 1: SPI 框架.....     | 8  |
| 图 2: 模块发起请求.....     | 12 |
| 图 3: SPI 从机发起请求..... | 13 |

Quectel  
Confidential

# 1 引言

移远通信 AG35-CET 和 AG35-EUT 模块支持 QuecOpen®方案。QuecOpen®是基于 Linux 的嵌入式开发平台，可简化 IoV 应用的软件设计和开发过程。有关 QuecOpen®的详细信息，请参考文档 [1]。

本文档适用于 SDK 构建环境的 QuecOpen®方案，主要介绍 AG35-CET 和 AG35-EUT 模块的 SPI 功能开发，包含 SPI 接口引脚定义和使用、设备树配置、SPI 设备驱动、编译示例及功能验证、驱动调试的方法等。

Quectel  
Confidential

## 2 SPI 说明

AG35-CET 和 AG35-EUT 模块默认提供 2 路 SPI 接口,支持主从模式,支持的最大时钟频率为 26 MHz。

模块的 SPI 框架有如下三个组成部分, 用户仅需修改 SPI 设备驱动:

- **SPI 核心**

SPI 核心提供了 SPI 总线驱动和设备驱动的注册/注销方法、SPI 通信方法、与具体控制器无关的代码以及检测设备的代码、检测设备地址的上层代码等。

- **SPI 总线驱动 (SPI 控制器)**

SPI 总线驱动是 SPI 控制器的软件实现, 由 CPU 控制, 可使 SPI 控制器与从设备完成数据通信。

- **SPI 设备驱动**

即客户的 SPI 从设备驱动, 是对 SPI 硬件体系结构中设备端的实现; 设备挂载在受 CPU 控制的 SPI 控制器上, 通过 SPI 控制器与 CPU 交换数据。

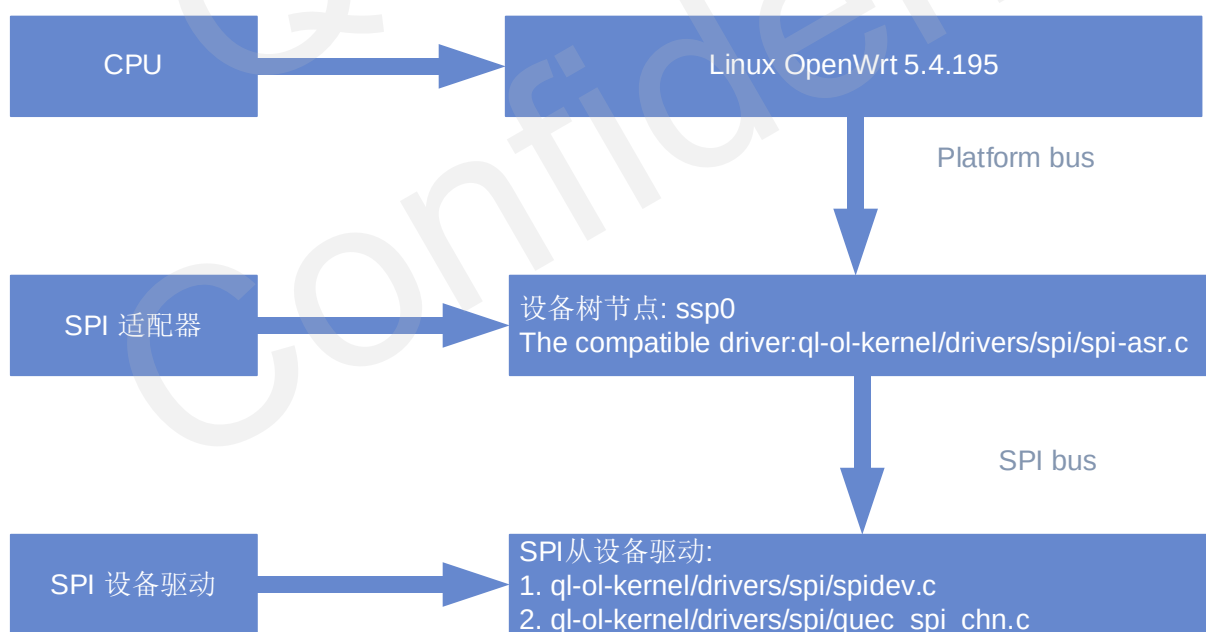


图 1: SPI 框架

其中 SPI adapter 为 SPI 控制器的设备树配置及驱动; SPI 从设备驱动为 SDK 提供的 SPI 设备驱动,

分为标准 4 线和扩展 6 线两种：

- 4 线：通常用来连接 SPI Flash、LCD 等，由模块发起请求；
- 6 线：通常用来与 MCU 通信，模块、MCU 均可发起请求，相比串口通信更高速。

Quectel  
Confidential

## 3 引脚定义及使用

### 3.1. 引脚定义

QuecOpen 方案下，模块默认提供 2 路 SPI：SPI1 和 SPI2。引脚定义如下表。

表 1：SPI1（软件设备树节点：ssp1）引脚定义

| 引脚名       | 引脚号 | I/O | 复用功能 0  |
|-----------|-----|-----|---------|
| SPI1_CLK  | 80  | DO  | GPIO_21 |
| SPI1_CS   | 79  | DO  | GPIO_22 |
| SPI1_DIN  | 78  | DIO | GPIO_24 |
| SPI1_DOUT | 77  | DIO | GPIO_23 |

表 2：SPI2（软件设备树节点：ssp0）引脚定义

| 引脚名       | 引脚号 | I/O | 复用功能 0  |
|-----------|-----|-----|---------|
| SPI2_CLK  | 195 | DO  | GPIO_33 |
| SPI2_CS   | 192 | DO  | GPIO_34 |
| SPI2_DIN  | 193 | DIO | GPIO_35 |
| SPI2_DOUT | 194 | DIO | GPIO_36 |

模块支持复用如下 GPIO 为 SPI。

表 3：复用 SPI（软件设备树节点：ssp2）引脚定义

| 引脚名   | 引脚号 | I/O | 复用功能 1   |
|-------|-----|-----|----------|
| GPIO5 | 147 | DO  | SSP2_CLK |
| GPIO6 | 150 | DO  | SSP2_FRM |
| GPIO7 | 159 | DO  | SSP2_TXD |
| GPIO8 | 143 | DI  | SSP2_RXD |

#### 备注

1. 非默认的复用功能需配置后方可生效。有关 SPI 复用功能的详情，请参考文档 [2]。
2. SSP2 接口可以定义成 SPI；当用作 SPI 时，仅支持主模式。

## 3.2. 引脚使用

### 3.2.1. 标准 4 线 SPI 定义

以 SPI1 为例，其对应的功能描述说明如下：

表 4：SPI1（软件设备树节点：ssp1）引脚定义

| 引脚名       | 引脚号 | I/O | 描述       | 备注               |
|-----------|-----|-----|----------|------------------|
| SPI1_CLK  | 80  | DO  | SPI 时钟   | 1.8 V 电压域，不用则悬空。 |
| SPI1_CS   | 79  | DO  | SPI 片选   |                  |
| SPI1_DIN  | 78  | DIO | SPI 数据输入 |                  |
| SPI1_DOUT | 77  | DIO | SPI 数据输出 |                  |

### 3.2.2. 扩展 6 线 SPI 使用

#### 3.2.2.1. 引脚使用

以 SPI1 为例，其 SPI 对应的功能描述说明如下：

表 5：扩展 6 线 SPI 引脚功能描述

| 引脚名       | 引脚号                             | I/O | 描述                                               | 备注               |
|-----------|---------------------------------|-----|--------------------------------------------------|------------------|
| SPI1_CLK  | 80                              | DO  | SPI 时钟                                           |                  |
| SPI1_CS   | 79                              | DO  | SPI 片选                                           | 1.8 V 电压域，不用则悬空。 |
| SPI1_DIN  | 78                              | DIO | SPI 数据输入                                         |                  |
| SPI1_DOUT | 77                              | DIO | SPI 数据输出                                         |                  |
| SPI_SRDY  | 可由用户选择<br>(默认选择引脚 62, 即 GPIO3)  | DI  | SPI 从机 ready 信号，空闲为低；当 SPI 从机准备好接收或发送数据时，该引脚被拉高。 |                  |
| SPI_MRDY  | 可由用户选择<br>(默认选择引脚 144, 即 GPIO4) | DO  | 模块输出信号，空闲为低；当模块要输出数据时，驱动自动拉高该引脚。                 |                  |

#### 3.2.2.2. 请求发起流程

##### ● 模块发起请求：

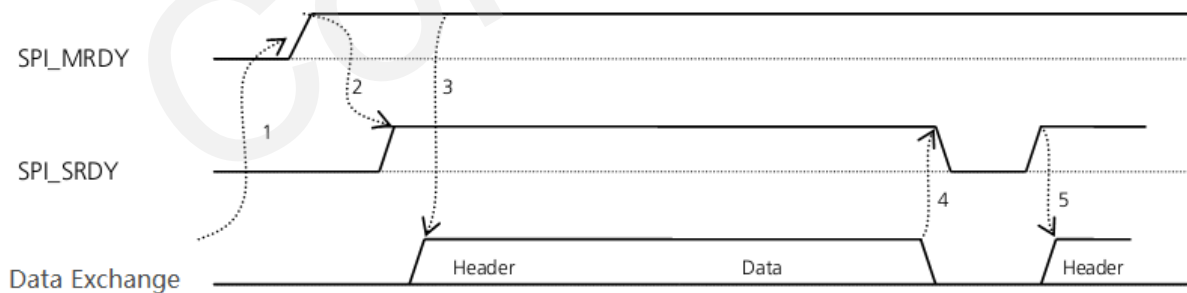


图 2：模块发起请求

模块发送数据流程：

1. 驱动自动拉高 SPI\_MRDY 通知 SPI 从机。

2. 判断 SPI\_SRDY 是否为高电平，若不为高电平则等待 SPI\_SRDY 的上升沿中断。
3. 收到 SPI\_SRDY 上升沿中断，开始 SPI 传输。
4. 传输完毕，若要继续发送数据，则保持 SPI\_MRDY 为高电平，并继续第 2 步，否则拉低 SPI\_MRDY。

SPI 从机接收数据流程：

1. 收到 SPI\_MRDY 上升沿中断，表示模块需要发送数据。
2. 准备好 SPI 传输，并拉高 SPI\_SRDY 通知模块开始 SPI 传输。
3. 等待 SPI 传输结束，然后拉低 SPI\_SRDY。
4. 若 SPI\_MRDY 为高电平，继续第 2 步。

#### ● SPI 从机发起请求：

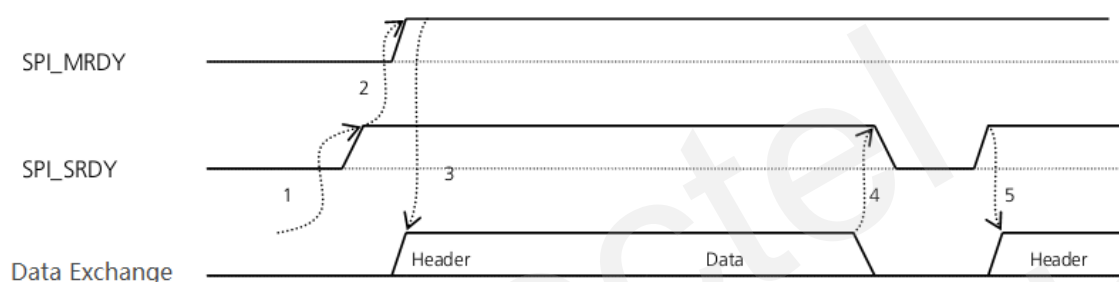


图 3：SPI 从机发起请求

SPI 从机发送数据流程：

1. 准备好 SPI 传输，并拉高 SPI\_SRDY。
2. 等待 SPI 传输结束，然后拉低 SPI\_SRDY。
3. 若要继续发送数据，则继续第 1 步。

模块接收数据流程：

1. 收到 SPI\_SRDY 上升沿中断，表示 SPI slave 需要发送数据。
2. 拉高 SPI\_MRDY，并开始 SPI 传输。
3. 等待传输结束，然后拉低 SPI\_MRDY。

## 4 设备树配置

### 4.1. SPI1 设备树配置

#### 4.1.1. SPI1 总线设备树配置

1. SPI1 总线的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/asr1806.dtsi`, SPI1 的设备树节点为 `ssp1`; SPI1 总线的默认配置如下所示:

```
ssp1: spi@d401b800 {
    compatible = "asr,asr-spi";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0xd401b800 0x30>;

    dmas = <&pdma0 54 0
           &pdma0 55 0>;
    dma-names = "rx", "tx";

    asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>;
    asr,ssp-clock-rate = <13000000>;
    asr,ssp-id = <2>;
    interrupts = <2>;
    asr,ssp-enhancement;
    asr,ssp-disable-dma;
    /* asr,ssp-hold-frame-low; */
    /* asr,spi-master-rxto = <3000>; */
    /* asr,spi-slave-rxto = <262144>; */
    /* asr,spi-pio-interval = <5>; */
    clocks = <&soc_clocks ASR1803_CLK_SSP1>;
    status = "disabled";
};
```

#### 备注

如上默认配置已经在 SPI 总线配置文件中设置完成, 用户无需重新设置。

2. SPI1 总线开关的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-common.dtsi`; SPI1 总线默认使能, 如下所示:

```

sspi: spi@d401b800 {
    status = "okay";
    compatible = "asr,asr-spi";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0xd401b800 0x30>;
    /* dmas = <&pdma0 54 0
        &pdma0 55 0>;
    dma-names = "rx", "tx"; */
    asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>;
    asr,ssp-clock-rate = <26000000>;
    asr,ssp-id = <2>;
    interrupts = <2>;
    asr,ssp-disable-dma;
    /* asr,ssp-hold-frame-low */
    clocks = <&soc_clocks ASR1803_CLK_SSP1>;
    pinctrl-names = "default";
    pinctrl-0 = <&sspi_pmx_func>;
    asr,sbi-inc-mode;
};

```

如需将 SPI1 接口引脚用作普通 GPIO 功能，需将 `status = "okay"` 改为 `status = "disabled"` 以关闭 SPI 总线。

#### 4.1.2. SPI1 引脚设备树配置

SPI1 引脚的设备树配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-common.dtsi`；其默认配置如下所示：

```

sspi_pmx_func: sspi_pmx_func {
    pinctrl-single,pins = <
        GPIO23 AF7 /* TXD */
        GPIO24 AF7 /* RXD */
        GPIO22 AF7 /* FRM */
        GPIO21 AF7 /* SCLK */
    >;
    DS_MEDIUM;PULL_NONE;EDGE_NONE;SL_NORMAL;
};

```

#### 备注

如上默认配置已经在 SPI 引脚设备树配置文件中设置完成，用户无需重新设置。若对引脚电平有特殊要求，则可根据具体需求对 SPI 引脚配置进行相应修改。

## 4.2. SPI2 设备树配置

### 4.2.1. SPI2 总线设备树配置

1. SPI2 总线的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/asr1806.dtsi`, SPI2 的设备树节点为 `ssp0`; SPI2 总线的默认配置如下所示:

```
ssp0: spi@d401b000 {
    compatible = "asr,asr-spi";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0xd401b000 0x30>;

    dmas = <&pdma0 52 0
           &pdma0 53 0>;
    dma-names = "rx", "tx";

    asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>;
    asr,ssp-clock-rate = <13000000>;
    asr,ssp-id = <1>;
    interrupts = <3>;
    asr,ssp-enhancement;
    asr,ssp-disable-dma;
    /* asr,ssp-hold-frame-low; */
    /* asr,spi-master-rxto = <3000>; */
    /* asr,spi-slave-rxto = <262144>; */
    /* asr,spi-pio-interval = <5>; */
    clocks = <&soc_clocks ASR1803_CLK_SSP0>;
    status = "disabled";
};
```

#### 备注

如上默认配置已经在 SPI 总线配置文件中设置完成, 用户无需重新设置。

2. SPI2 总线开关的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-common.dtsi`; SPI2 总线默认使能, 如下所示:

```
ssp0: spi@d401b000 {
    status = "okay";
    /* dmas = <&pdma0 52 0
       &pdma0 53 0>;
    dma-names = "rx", "tx"; */
    pinctrl-names = "default";
    pinctrl-0 = <&ssp0_pmx_func>;
    asr,spi-inc-mode;
};
```

如需将 SPI2 接口引脚用作普通 GPIO 功能，需将 `status = "okay"` 改为 `status = "disabled"` 以关闭 SPI 总线。

### 4.2.2. SPI2 引脚设备树配置

SPI2 引脚的设备树配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-common.dtsi`；其默认配置如下所示：

```
ssp0_pmx_func: ssp0_pmx_func {
    pinctrl-single,pins = <
        GPIO36 AF1 /* TXD */
        GPIO35 AF1 /* RXD */
        GPIO34 AF1 /* FRM */
        GPIO33 AF1 /* SCLK */
    >;
    DS_MEDIUM;PULL_NONE;EDGE_NONE;SL_NORMAL;
};
```

#### 备注

如上默认配置已经在 SPI 引脚设备树配置文件中设置完成，用户无需重新设置。若对引脚电平有特殊要求，则可根据具体需求对 SPI 引脚配置进行相应修改。

### 4.3. 复用 SPI 设备树配置

1. 复用 SPI 配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/asr1806.dtsi`，复用的设备树节点为 `ssp2`；复用 SPI 的默认配置如下所示：

```
ssp2: spi@d401c000 {
    compatible = "asr,asr-spi";
    #address-cells = <1>;
    #size-cells = <0>;
    reg = <0xd401c000 0x30>;

    dmas = <&pdma0 60 0
           &pdma0 61 0>;
    dma-names = "rx", "tx";

    asr,ssp-lpm-qos = <PM_QOS_CPUIDLE_BLOCK_AXI>;
    asr,ssp-clock-rate = <13000000>;
    asr,ssp-id = <3>;
    interrupts = <1>;
    asr,ssp-enhancement;
    /* asr,ssp-disable-dma; */
    /* asr,ssp-hold-frame-low; */
    /* asr,spi-master-rxto = <3000>; */
    /* asr,spi-slave-rxto = <262144>; */
    /* asr,spi-pio-interval = <5>; */
    clocks = <&soc_clocks ASR1803_CLK_SSP2>;
    status = "disabled";
};
```

## 备注

如上默认配置已经在 SPI 总线配置文件中设置完成，用户无需重新设置。

2. 复用 SPI 开关的配置见 SDK 文件 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-common.dtsi`；复用引脚默认关闭，如下所示：

```
ssp2: spi@d401c000 {
    status= "disabled";
    asr,ssp-id = <3>;
    /* dmas = <&pdma0 60 0
           &pdma0 61 0>;
    dma-names = "rx", "tx"; */
    pinctrl-names = "default";
    pinctrl-0 = <&ssp2_pmx_func>;
    asr,spi-inc-mode;
};
```

如需将普通 GPIO 复用为 SPI 外设功能，需将 `status = "disabled"` 改为 `status = "okay"` 以打开 SPI 总线。

## 备注

如上默认配置已经在 SPI 引脚设备树配置文件中设置完成，用户无需重新设置。

# 5 SPI 设备驱动

模块提供两种 SPI 设备驱动，分别为标准 4 线和扩展 6 线。SPI 设备驱动以内核模块方式编译，编译后的 SPI 设备驱动文件存放于 `/lib/modules/5.4.195/` 目录下。本章以 SPI1 为例。

## 5.1. 标准 4 线 SPI 设备驱动

驱动文件 `spidev.c` 位于 `ql-ol-kernel/drivers/spi/` 目录下。编译后的内核模块存放于 `rootfs` 的 `/lib/modules/5.4.195/` 目录下。执行 `insmod` 命令即可加载驱动。直接通过执行 `insmod` 命令传入参数比设备树传参方式更为灵活。内核模块加载时支持的参数如下：

表 6：内核模块加载时支持的参数（标准 4 线 SPI 设备驱动）

| 参数                  | 描述                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
|---------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|                     | SPI 控制器编号，由下图配置决定。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                     |
| <code>busnum</code> | <pre> ssp0: spi@d401b000 {     compatible = "asr,asr-spi";     #address-cells = &lt;1&gt;;     #size-cells = &lt;0&gt;;     reg = &lt;0xd401b000 0x30&gt;;      dmas = &lt;&amp;pdma0 52 0            &amp;pdma0 53 0&gt;;     dma-names = "rx", "tx";      asr,ssp-lpm-qos = &lt;PM_QOS_CPUIDLE_BLOCK_AXI&gt;;     asr,ssp-clock-rate = &lt;13000000&gt;;     asr,ssp-id = &lt;1&gt;;     interrupts = &lt;3&gt;;     asr,ssp-enhancement;     asr,ssp-disable-dma;     /* asr,ssp-slave-mode; */     /* asr,slave-rxtimer-to-ms = &lt;0&gt;; */     /* asr,ssp-hold-frame-low; */     /* asr,spl-master-rxto = &lt;8000&gt;; */     /* asr,spl-slave-rxto = &lt;262144&gt;; */     /* asr,spl-pio-interval = &lt;5&gt;; */     clocks = &lt;&amp;soc_clocks ASR1803_CLK_SSP0&gt;;     status = "disabled"; }; </pre> |

此参数必须传入，否则 SPI 设备驱动会注册失败。

|                   |                                                                                                                                                                                                                                                                                                       |
|-------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <i>chipselect</i> | 片选；取值支持 0、1、2 和 3。<br>此参数必须传入，否则 SPI 设备驱动会注册失败。                                                                                                                                                                                                                                                       |
| <i>spimode</i>    | SPI 工作模式。<br>默认值：SPI_MODE_3。<br>SPI 共支持 4 种工作模式：SPI_MODE_0、SPI_MODE_1、SPI_MODE_2、SPI_MODE_3，其值为相位（CPHA 0x01）和极性（CPOL 0x02）的按位或。用户在执行 <b>insmod</b> 命令传入参数时可进行修改。<br><br>时钟极性（CPOL）：定义 SPI 总线空闲时，时钟信号 SCLK 的电平（1：SCLK 为高电平；0：SCLK 为低电平）。<br>时钟相位（CPHA）：定义 SPI 数据采样是在 SCLK 的第几个边沿（0：第一个边沿开始；1：第二个边沿开始）。 |
| <i>maxspeed</i>   | 最大时钟频率；可选参数。<br>默认值：13000000；单位：Hz。<br>该参数支持的值包括：812500、1000000、1625000、3250000、6500000、13000000 和 26000000；实际支持的最大值由 SPI 控制器的配置决定，与理论最大值可能不同。                                                                                                                                                        |
| <i>bufsiz</i>     | 缓存大小；可选参数。<br>默认值：4096，单位：字节。<br>用户可根据每次传输的数据量大小来设定该参数。                                                                                                                                                                                                                                               |

用户可执行以下 **insmod** 命令加载 SPI 驱动：

```
insmod /lib/modules/5.4.195/spidev.ko busnum=1 chipselect=0 spimode=0 maxspeed=6500000
```

加载成功时，使用 **lsmod** 命令可查看到 *spidev* 字段，如下图所示。

```
pppox      16384  1 pppoe
slhc       16384  1 ppp_generic
spidev     16384  0
tunnel4    16384  2 xfrm4_tunnel,ipip
tunnel6    16384  2 xfrm6_tunnel,ip6_tunnel
xfrm_algo  16384  7 esp6,ah6,esp4,ah4,xfrm_user,xfrm_ipcomp,af_key
xfrm_ipcomp 16384  2 ipcomp6,ipcomp
```

## 5.2. 扩展 6 线 SPI 设备驱动

驱动文件 *quec\_spi\_chn.c* 位于 *ql-ol-kernel/drivers/spi/*目录下。编译后的内核模块存放于 *rootfs* 的 */lib/modules/5.4.195/*目录下。执行 **insmod** 命令即可加载驱动。直接通过执行 **insmod** 命令传入参数比设备树传参方式更为灵活。内核模块加载时支持的参数如下：

表 7：内核模块加载时支持的参数（扩展 6 线 SPI 设备驱动）

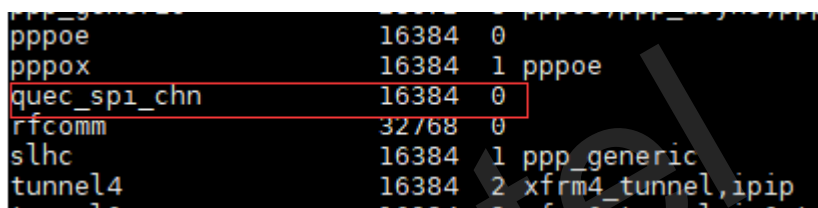
| 参数         | 描述                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
|------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
|            | SPI 控制器编号，由下图配置决定。                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                      |
| busnum     | <pre> ssp0: spi@d401b000 {     compatible = "asr,asr-spi";     #address-cells = &lt;1&gt;;     #size-cells = &lt;0&gt;;     reg = &lt;0xd401b000 0x30&gt;;      dmas = &lt;&amp;pdma0 52 0            &amp;pdma0 53 0&gt;;     dma-names = "rx", "tx";      asr,ssp-lpm-qos = &lt;PM_QOS_CPUIDLE_BLOCK_AXI&gt;;     asr,ssp-clock-rate = &lt;13000000&gt;;     asr,ssp-id = &lt;1&gt;;     interrupts = &lt;3&gt;;     asr,ssp-enhancement;     asr,ssp-disable-dma;     /* asr,ssp-slave-mode; */     /* asr,slave-rxtimer-to-ms = &lt;0&gt;; */     /* asr,ssp-hold-frame-low; */     /* asr,spi-master-rxto = &lt;8000&gt;; */     /* asr,spi-slave-rxto = &lt;262144&gt;; */     /* asr,spi-pio-interval = &lt;5&gt;; */     clocks = &lt;&amp;soc_clocks ASR1803_CLK_SSP0&gt;;     status = "disabled"; }; </pre> <p>此参数必须传入，否则 SPI 设备驱动会注册失败。</p> |
| chipselect | <p>片选；取值支持 0、1、2 和 3。</p> <p>此参数必须传入，否则 SPI 设备驱动会注册失败。</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| spi_mode   | <p>SPI 工作模式。</p> <p>默认值：SPI_MODE_0。</p> <p>SPI 共支持 4 种工作模式：SPI_MODE_0、SPI_MODE_1、SPI_MODE_2、SPI_MODE_3，其值为相位（CPHA 0x01）和极性（CPOL 0x02）的按位或。用户在执行 insmod 命令传入参数时可进行修改。</p> <p><b>时钟极性（CPOL）：</b>定义 SPI 总线空闲时，时钟信号 SCLK 的电平（1：SCLK 为高电平；0：SCLK 为低电平）。</p> <p><b>时钟相位（CPHA）：</b>定义 SPI 数据采样是在 SCLK 的第几个边沿（0：第一个边沿开始；1：第二个边沿开始）。</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| speed_hz   | <p>最大时钟频率；可选参数。</p> <p>默认值：13000000；单位：Hz。</p> <p>该参数支持的值包括：812500、1000000、1625000、3250000、6500000、13000000 和 26000000；实际支持的最大值由 SPI 控制器的配置决定，与理论最大值可能不同。</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| frame_size | <p>缓存大小；可选参数。</p> <p>默认值：512，单位：字节。</p> <p>用户可根据每次传输的数据量的大小来设定该参数。</p>                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |

|                       |                                            |
|-----------------------|--------------------------------------------|
| <i>gpiomodemready</i> | SPI_MRDY 引脚。<br>默认值：120。对应模块的引脚为 GPIO_120。 |
| <i>gpiomcuready</i>   | SPI_SRDY 引脚。<br>默认值：122。对应模块的引脚为 GPIO_122。 |

可执行以下 **insmod** 命令加载 SPI 驱动：

```
insmod /lib/modules/5.4.195/quec_spi_chn.ko busnum=1 chipselect=0 spi_mode=0
speed_hz=6500000 gpiomcuready=120 gpiomodemready=122
```

加载成功时，使用 **lsmod** 命令可查看到 *quec\_spi\_chn* 字段，如下图所示。



```
pppoe 16384 0
pppox 16384 1 pppoe
quec_spi_chn 16384 0
rfcomm 32768 0
slhc 16384 1 ppp_generic
tunnel4 16384 2 xfrm4_tunnel,ipip
tunnel6 16384 2 xfrm6_tunnel,ip6t...
```

## 备注

不使用 SDK 提供的 SPI 设备驱动时，用户需从外设设备供应商处获取设备驱动和配置手册，并按照配置手册自行进行相关配置。

## 6 编译示例及功能验证

模块 SDK 包中包含了 SPI 功能的编译示例 *main.c*，该示例路径为 *sample/spi/*。本章所述示例演示初始化 SPI 设备，然后读取数据。

### 6.1. 编译步骤

#### 6.1.1. 标准 4 线 SPI 编译步骤

步骤一：初始化 SPI 设备，并进行 SPI 数据传输。如下所示，*sample/spi/main.c* 示例以工作模式 SPI \_MODE\_0、8 位字长、6.5 MHz 最大时钟频率初始化 SPI 设备，并向设备写入 1024 个字节，同时读取 1024 字节；用户可根据实际应用需求编译相关信息。

```
int main(int argc, char *argv[])
{
    int fd;
    int i;
    uint8_t writebuf[1024];
    uint8_t readbuf[1024];

    fd = ql_spi_init(device, SPI_MODE_0, 8, S_6_5M);

    for(i = 0 ; i < 1024; i++)
        writebuf[i] = i%256;

    ql_spi_write_read(fd, writebuf, readbuf, 1024);

    for (i = 0; i < 1024; i++) {
        if (!(i % 32))
            puts("");
        printf("%.2X ", readbuf[i]);
    }
    puts("");

    ql_spi_deinit(fd);
    return 0 ;
}
```

步骤二：进入 *sample/spi/* 目录，执行 **make** 命令生成可执行文件 *spi\_test*。

```
ql@ubuntu:~/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/sample/spi$ make
/opt/ql_crosstools/ql-ag35-1806e-gcc-8.4.0-glibc-v1-toolchain/bin/arm-openwrt-linux-gcc -o main.o -c main.c -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -I. -I/home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-sysroots/usr/include -I/home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-sysroots/usr/include/ql-sdk -Wall -Wundef
/opt/ql_crosstools/ql-ag35-1806e-gcc-8.4.0-glibc-v1-toolchain/bin/arm-openwrt-linux-gcc -o spi_test main.o -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -L. -L/home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-sysroots/usr -L/home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-sysroots/usr/lib -lql_sdk -lpthread -lql_sys_log -lql_lib_ipc -lql_lib_utils
```

### 6.1.2. 扩展 6 线 SPI 编译步骤

进入 `sample/spi_six_line/` 目录，执行 **make** 命令生成可执行文件 `example_six_line_spi`。

```
dawn@ubuntu:~/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/sample/spi_six_line$
dawn@ubuntu:~/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/sample/spi_six_line$ make
/opt/ql_crosstools/ql-ag35-1806e-gcc-8.4.0-glibc-v1-toolchain/bin/arm-openwrt-linux-gcc -o main.o -c main.c -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -I. -I/home/dawn/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/ql-sysroots/usr/include -I/home/dawn/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/ql-sysroots/usr/include/ql-sdk -Wall -Wundef
/opt/ql_crosstools/ql-ag35-1806e-gcc-8.4.0-glibc-v1-toolchain/bin/arm-openwrt-linux-gcc -o example_six_line_spi main.o -march=armv7-a -marm -mfp=neon -mfloat-abi=hard -L. -L/home/dawn/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/ql-sysroots/usr -L/home/dawn/ql-ol-extsdk-ag35cetcar01a01m2g_ocpu/ql-sysroots/usr/lib -lql_sdk -lpthread -lql_sys_log -lql_lib_ipc -lql_lib_utils
```

## 6.2. 功能验证

### 6.2.1. SPI1 主模式功能验证

步骤一：进入 `sample/spi/` 目录，修改 `main.c` 文件如下，执行 **make** 命令生成可执行文件 `spi_test`。

```
#include <linux/types.h>
#include <linux/spi/spidev.h>

#define device "/dev/spidev2.0"

typedef enum
{
    SPI_MODE_0 = SPI_MODE_0,
    SPI_MODE_1 = SPI_MODE_1,
    SPI_MODE_2 = SPI_MODE_2,
    SPI_MODE_3 = SPI_MODE_3,
} SPI_MODE;
```

步骤二：执行如下命令上传 `spi_test` 至模块。

```
adb push <spi_test 在上位机路径> <模块内部路径，如/tmp>
```

步骤三：执行如下命令修改文件权限。

```
chmod 777 spi_test
```

步骤四：在 LTE OPEN EVB 上，短接 GPIO\_20 和 GPIO\_21。

步骤五：执行如下命令动态加载 SPI 设备驱动。

```
insmod /lib/modules/5.4.195/spidev.ko busnum=2 chipselect=0 spimode=0 maxspeed=6500000
```

步骤六：执行 `ls /dev/spi*` 查看 spidev2.0 设备是否生成。若返回 `/dev/spidev2.0` 表示已生成 spidev2.0 设备。

步骤七：执行 `spi_test`。若收到的数据与发出的数据一致，则 SPI 编译成功，功能验证通过。

```
root@openWrt:/data# ls /dev/spidev2.0
/dev/spidev2.0
root@openWrt:/data# ./spi_test
< open(/dev/spidev2.0, 0_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)

00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
root@openWrt:/data#
```

## 6.2.2. SPI2 主模式功能验证

参考第 6.1.1 章编译并生成可执行文件 `spi_test` 后，可参考如下所述步骤完成功能验证。

步骤一：执行如下命令上传 `spi_test` 至模块。

```
adb push <spi_test 在上位机路径> <模块内部路径，如/tmp/>
```

步骤二：执行如下命令修改文件权限。

```
chmod 777 spi_test
```

步骤三：在 LTE OPEN EVB 上短接 SPI\_MOSI\_AG35（J1005）和 SPI\_MISO\_AG35（J1005）。

步骤四：执行如下命令动态加载 SPI 设备驱动。

```
insmod /lib/modules/5.4.195/spidev.ko busnum=1 chipselect=0 spimode=0 maxspeed=6500000
```

步骤五：执行 `ls /dev/spi*` 查看 `spidev1.0` 设备是否生成。若返回 `/dev/spidev1.0` 表示已生成 `spidev1.0` 设备。

步骤六：执行 `spi_test`。若收到的数据与发出的数据一致，则 SPI 编译成功，功能验证通过。

```
root@OpenWrt:/tmp#
root@OpenWrt:/tmp# ./spi_test
< open(/dev/spidev1.0, 0_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed      : 6500000 Hz (6500 KHz)

00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
root@OpenWrt:/tmp#
```

### 6.2.3. SPI 从模式功能验证

本章节介绍主从模式的配置及从模式功能验证。设置 SPI1 为主模式，SPI2 为从模式。

**步骤一：**修改 `ql-ol-kernel/arch/arm/boot/dts/ql-1806e-std.dts`，宏定义配置成 0 为主模式，配置成 1 为从模式。现将 SPI2 设置为从模式。

```
#define CONFIG_USE_ES8311    0

/* Config spi1 or spi2 slave mode */
#define CONFIG_SPI1_SLAVE    0
#define CONFIG_SPI2_SLAVE    1

#define CONFIG_USE_FCS950U    0
```

**步骤二：**复制 `ql-ol-kernel/drivers/spi/`目录下的 `spidev.c` 文件并重命名为 `spidev1.c`，修改 `spidev1.c` 文件如下：

```
/*
 * particular SPI bus or device.
 */
#define SPIDEV_MAJOR        154    /* assigned */
#define N_SPI_MINORS        32     /* ... up to 256 */

static DECLARE_BITMAP(minors, N_SPI_MINORS);
```

```
/* The main reason to have this class is to make mdev/udev create the
 * /dev/spidevB.C character device nodes exposing our userspace API.
 * It also simplifies memory management.
 */

static struct class *spidev_class;

#ifdef CONFIG_OF
static const struct of_device_id spidev_dt_ids[] = {
    { .compatible = "asr,slic" },
    {}
};
MODULE_DEVICE_TABLE(of, spidev_dt_ids);
#endif
```

```
static struct spi_driver spidev_spi_driver = {
    .driver = {
        .name = "spidev1",
        .of_match_table = of_match_ptr(spidev_dt_ids),
        .acpi_match_table = ACPI_PTR(spidev_acpi_ids),
    },
    .probe = spidev_probe,
    .remove = spidev_remove,

    /* NOTE: suspend/resume methods are not necessary here.

```

```
static int __init spidev_init(void)
{
    int status;

    /* Claim our 256 reserved device numbers. Then register a class
     * that will key udev/mdev to add/remove /dev nodes. Last, register
     * the driver which manages those device numbers.
     */
    BUILD_BUG_ON(N_SPI_MINORS > 256);
    status = register_chrdev(SPIDEV_MAJOR, "spi1", &spidev_fops);
    if (status < 0)
        return status;

    spidev_class = class_create(THIS_MODULE, "spidev1");
    if (IS_ERR(spidev_class)) {
        unregister_chrdev(SPIDEV_MAJOR, spidev_spi_driver.driver.name);
        return PTR_ERR(spidev_class);
    }

    status = spi_register_driver(&spidev_spi_driver);
    if (status < 0) {
        class_destroy(spidev_class);
        unregister_chrdev(SPIDEV_MAJOR, spidev_spi_driver.driver.name);
    }

    //jayde add ,20220426: Users by parameter tests
    if (busnum != -1 && chipselect != -1) {
        struct spi_board_info chip = {
            .modalias = "spidev1",
            .mode = SPI_MODE,

```

步骤三：修改 `ql-ol-kernel/drivers/spi/` 目录下的 `Makefile` 文件如下：

```
# small core, mostly translating board-specific
# config declarations into driver model code
obj-$(CONFIG_SPI_MASTER) += spi.o
obj-$(CONFIG_SPI_MEM) += spi-mem.o
obj-$(CONFIG_SPI_SPIDEV) += spidev.o
obj-$(CONFIG_SPI_SPIDEV) += spidev1.o
#2022/05/05 jayden add, compatible with spi 6-line driver
obj-$(CONFIG_SPI_SPIDEV_6LINE) += quec_spi_chn.o
obj-$(CONFIG_SPI_LOOPBACK_TEST) += spi-loopback-test.o

```

步骤四：升级镜像文件，**push** 驱动 *spidev1.ko* 到模块。

1. 在根目录下执行 **make** 编译，使用 */target/* 目录下生成的 *zImage* 和 *root.squashfs* 镜像文件替换固件包 */update/* 目录下的同名文件，在上位机使用 *SWDownloader.exe* 工具进行升级，有关工具使用的详情，请参考文档 [3]。

```
ql@ubuntu:~/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu$ make
if [ ! -f /home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-kernel/.config ]; then cp -f /home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-kernel/deconfig_asr1806p301 /home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-kernel/.config; fi
make -C /home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-kernel ARCH=arm CROSS_COMPILE=/opt/ql_crosstools/ql-ag35-1806e-gcc-8.4.0-glibc-v1-toolchain/bin/arm-openwrt-linux- HOSTCFLAGS="-O2 -Wall -Wmissing-prototypes -Wstrict-prototypes" KBUILD_HAVE_NLS=no -j4 zImage dtbs
make[1]: Entering directory '/home/ql/sdk/ql-ol-extsdk-ag35cetcar01a02m2g_ocpu/ql-ol-kernel'
HOSTCC scripts/basic/fixdep
```

2. 将 *ql-ol-kernel/drivers/spi/* 路径下的 *spidev1.ko* 驱动 **push** 到模块 */dev/* 路径下。

```
adb push <spidev1.ko 在上位机路径> /dev/
```

步骤五：执行文件

1. 进入 *sample/spi/* 目录，执行 **make** 命令生成可执行文件 *spi\_test*，重命名可执行文件 *spi\_test* 为 *spi\_test1*。
2. 进入 *sample/spi/* 目录，修改 *main.c* 文件如下。执行 **make** 命令生成可执行文件 *spi\_test*，重命名可执行文件 *spi\_test* 为 *spi\_test2*。

```
#include <linux/types.h>
#include <linux/spi/spidev.h>

#define device "/dev/spidev2.0"

typedef enum
{
    SPI_MODE0 = SPI_MODE_0,
    SPI_MODE1 = SPI_MODE_1,
    SPI_MODE2 = SPI_MODE_2,
    SPI_MODE3 = SPI_MODE_3,
}SPI_MODE;
```

步骤六：执行如下命令上传 *spi\_test1*、*spi\_test2* 至模块。

```
adb push <spi_test 在上位机路径> <模块内部路径，如/tmp/>
```

步骤七：执行如下命令修改文件权限。

```
chmod 777 spi_test1
chmod 777 spi_test2
```

步骤八：在 LTE OPEN EVB 上：

- 短接 GPIO\_23 (J0201) 和 SPI\_CLK\_AG35 (J1005)；
- 短接 GPIO\_22 (J0201) 和 SPI\_CS\_N\_AG35 (J1005)；
- 短接 GPIO\_21 (J0201) 和 SPI\_MOSI\_AG35 (J1005)；
- 短接 GPIO\_20 (J0201) 和 SPI\_MISO\_AG35 (J1005)；

步骤九：执行如下命令动态加载 SPI 通用设备驱动。

```
insmod /lib/modules/5.4.195/spidev.ko busnum=1 chipselect=0 spimode=0 maxspeed=6500000
insmod /dev/spidev1.ko busnum=2 chipselect=0 spimode=0 maxspeed=6500000
```

步骤十：执行如下命令查看 *spidev1.0*、*spidev2.0* 设备是否生成。若返回 */dev/spidev1.0*、*/dev/spidev2.0* 表示已生成 *spidev1.0*、*spidev2.0* 设备。

```
# ls /dev/spidev1.0
/dev/spidev1.0
# ls /dev/spidev2.0
/dev/spidev2.0
```

步骤十一：先在模块上执行 *spi\_test1* 准备接收数据，然后在上位机执行 *spi\_test2* 发送数据。若收到的数据与发出的数据一致，则 SPI 编译成功，功能验证通过。

```
/tmp #
/tmp # ./spi_test1
./spi_test1
X open(/dev/spidev1.0, O_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
```

```
root@openwrt:/tmp#
root@openwrt:/tmp# ./spi_test2
< open(/dev/spidev2.0, O_RDWR)=3 >
spi mode:0x0
bits per word: 8
max speed : 6500000 Hz (6500 KHz)
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
00 01 02 03 04 05 06 07 08 09 0A 0B 0C 0D 0E 0F 10 11 12 13 14 15 16 17 18 19 1A 1B 1C 1D 1E 1F
20 21 22 23 24 25 26 27 28 29 2A 2B 2C 2D 2E 2F 30 31 32 33 34 35 36 37 38 39 3A 3B 3C 3D 3E 3F
40 41 42 43 44 45 46 47 48 49 4A 4B 4C 4D 4E 4F 50 51 52 53 54 55 56 57 58 59 5A 5B 5C 5D 5E 5F
60 61 62 63 64 65 66 67 68 69 6A 6B 6C 6D 6E 6F 70 71 72 73 74 75 76 77 78 79 7A 7B 7C 7D 7E 7F
80 81 82 83 84 85 86 87 88 89 8A 8B 8C 8D 8E 8F 90 91 92 93 94 95 96 97 98 99 9A 9B 9C 9D 9E 9F
A0 A1 A2 A3 A4 A5 A6 A7 A8 A9 AA AB AC AD AE AF B0 B1 B2 B3 B4 B5 B6 B7 B8 B9 BA BB BC BD BE BF
C0 C1 C2 C3 C4 C5 C6 C7 C8 C9 CA CB CC CD CE CF D0 D1 D2 D3 D4 D5 D6 D7 D8 D9 DA DB DC DD DE DF
E0 E1 E2 E3 E4 E5 E6 E7 E8 E9 EA EB EC ED EE EF F0 F1 F2 F3 F4 F5 F6 F7 F8 F9 FA FB FC FD FE FF
```

## 6.2.4. 6 线 SPI 功能验证

扩展六线 SPI 的源码为 *ql-ol-kernel/drivers/spi/quec\_spi\_chn.c*。参考第 6.1.2 章编译完成并生可执行文件 *example\_six\_line\_spi* 后，可按照如下步骤进行功能验证。

步骤一. 执行如下命令上传 *example\_six\_line\_spi* 至模块。

```
adb push <example_six_line_spi 在上位机路径> <模块内部路径, 如/data/>
```

步骤二. 执行如下命令修改文件权限:

```
chmod 777 example_six_line_spi
```

步骤三. LTE OPEN EVB 上:

- 短接 SPI\_MOSI\_AG35 (J1005) 和 SPI\_MISO\_AG35 (J1005);
- 短接 GPIO3 (J0203) 和 GPIO4 (J0203)。

步骤四. 执行如下命令动态加载六线 SPI 设备驱动。

```
insmod /lib/modules/5.4.195/quec_spi_chn.ko busnum=1 chipselect=0 spi_mode=0
speed_hz=6500000 gpiomcuready=120 gpiomodemready=122
```

步骤五. 执行 `ls /dev/spi*` 查看 spi1\_0\_0 设备是否生成。若返回 `/dev/spi1_0_0` 表示已生成 spi1\_0\_0 设备。

```
## ls /dev/spi*
/dev/spi1_0_0 /dev/spi1_0_2 /dev/spi1_0_4 /dev/spi1_0_6
/dev/spi1_0_1 /dev/spi1_0_3 /dev/spi1_0_5 /dev/spi1_0_7
```

步骤六. 执行 *example\_six\_line\_spi*。若收到的数据与发出的数据一致, 则 SPI 编译成功, 功能验证通过。

```
root@OpenWrt:~# insmod /lib/modules/5.4.195/quec_spi_chn.ko busnum=1 chipselect
=0 spimode=0 maxspeed=19200000 gpiomcuready=120 gpiomodemready=122
root@OpenWrt:~#
root@OpenWrt:~#
root@OpenWrt:~# cat /sys/kernel/debug/gpio
gpiochip0: GPIOs 0-127, gpio-pxa:
gpio-8 ( |status-led ) out hi
gpio-20 ( |mdio-reset ) out hi
gpio-38 ( |sleep_sys_ind ) out hi
gpio-117 ( |mdio_ldo ) out hi
gpio-118 ( |wakeup_in ) in hi
gpio-119 ( |usim0 ) in lo
gpio-120 ( |SPI_SRDY ) in lo
gpio-122 ( |SPI_MRDY ) out lo
gpio-125 ( |gnss_gpio_power ) out lo
root@OpenWrt:~#
root@OpenWrt:~#
root@OpenWrt:~# cd /data
root@OpenWrt:/data# ./example_six_line_spi
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
read 16 bytes
0 1 2 3 4 5 6 7 8 9 a b c d e f
```

# 7 驱动调试

本章主要介绍 SPI 驱动的调试方法（以 SPI1 为例），帮助用户定位在模块运行期间出现的意外问题。

**步骤一：**确认 SPI 总线是否使能。若存在 `/spi1/` 文件夹，表示 SPI 总线已经使能，如下图所示：

```
root@openWrt:~#
root@openWrt:~# ls /sys/devices/platform/soc/d4000000.apb/asr-spi.0/spi_master
spi1
root@openWrt:~#
```

**步骤二：**执行如下命令，确认 SPI 引脚配置是否成功：

```
cat /sys/kernel/debug/pinctrl/d401e000.pinmux-pinctrl-single/pinmux-pins
```

若当前配置的 GPIO 已注册在 SPI 下，表示 SPI 引脚配置成功。

```
pin 87 (PIN87): d4036000.uart (GPIO UNCLAIMED) function gps_pmx_uart_rxd group gps_pmx_uart_rxd
pin 88 (PIN88): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 89 (PIN89): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 90 (PIN90): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 91 (PIN91): asr-spi.0 (GPIO UNCLAIMED) function ssp0_pmx_func group ssp0_pmx_func
pin 92 (PIN92): (MUX UNCLAIMED) (GPIO UNCLAIMED)
pin 93 (PIN93): quec_gpio_lpm (GPIO UNCLAIMED) function sleep_sys_ind group sleep_sys_ind
```

**步骤三：**打开 SPI 调试日志。一般通过 `kernel log` 即可查看 SPI 相关错误，通过现有的错误即可分析相关问题。

## 8 附录 参考文档及术语缩写

表 8：参考文档

| 文档名称                                                      |
|-----------------------------------------------------------|
| [1] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_快速开发指导        |
| [2] Quectel_AG35-CET&AG35-EUT_QuecOpen_GPIO_Configuration |
| [3] Quectel_AG35-CET&AG35-EUT_QuecOpen(SDK)_固件下载指导        |

表 9：术语缩写

| 缩写   | 英文全称                         | 中文全称    |
|------|------------------------------|---------|
| CPHA | Clock Phase                  | 时钟相位    |
| CPOL | Clock Polarity               | 时钟极性    |
| CPU  | Central Processing Unit      | 中央处理器   |
| CS   | Chip Select                  | 片选      |
| DI   | Digital Input                | 数字输入    |
| DO   | Digital Output               | 数字输出    |
| EVB  | Evaluation Board             | 评估板     |
| GPIO | General Purpose Input/Output | 通用输入/输出 |
| I/O  | Input/Output                 | 输入/输出   |
| IoV  | Internet of Vehicles         | 车联网     |
| LTE  | Long-Term Evolution          | 长期演进    |
| SCLK | Serial Clock                 | 串行时钟    |
| SDK  | Software Development Kit     | 软件开发工具包 |

SPI

Serial Peripheral Interface

串行外设接口

Quectel  
Confidential